



Hochschule
Bonn-Rhein-Sieg
University of Applied Sciences



Introduction to Linux

Foundation Course – Winter Semester 2026

September 10, 2026

Ujjwal Patil

What is GNU/Linux?

- **Free**, open-source, Unix-like operating system
- **Kernel** written by Linus Torvalds in 1991
- GNU tools + Linux kernel = **GNU/Linux**
- Many **distributions** (distros) available:
Ubuntu, Fedora, Debian, Arch, openSUSE ...



This course

Ubuntu 22.04 LTS (Jammy Jellyfish)

Why Linux for robotics?

ROS2 Humble is only officially supported on Ubuntu 22.04. Every robot you will program here runs Linux.

Linux is Everywhere

You already use it every day

- **Android** – every phone in your pocket
- **Servers** – powers most of the internet
- **Supercomputers** – 100% of TOP500
- **Smart TVs, routers, embedded systems**
- **Robotics** – ROS2 runs natively on Linux

Why it matters here

Learning the terminal is not optional. It is the primary interface for ROS2, robotics, and remote systems over SSH.

The GNU/Linux stack

Hardware → Kernel → GNU Tools → Shell
→ Applications



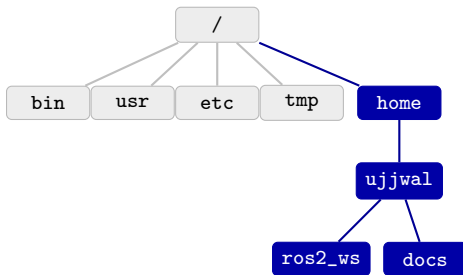
Linux Directory Tree

No drive letters – one single tree starting at / (root)

- Root is `/` – top of the tree
- System folders: **protected** (need sudo)
- **Your space**: `/home/username`
- `~` always means your home

Key system folders

<code>/bin</code>	essential programs
<code>/usr</code>	installed software
<code>/etc</code>	configuration files
<code>/tmp</code>	temporary files



Blue = your space. Gray = system (protected).

Files and Folders

A trailing / marks a folder (optional)

```
/home/ujjwal/projects/
```

Everything else is a file

```
/home/ujjwal/main.cpp
```

Paths are case sensitive!

`main.cpp` $\neq$ `Main.cpp`

Extension is part of the name:

`file.cpp` $\neq$ `file.txt`

Absolute path

Starts with / – works from anywhere

```
/home/ujjwal/code/hello.cpp
```

Relative path

From your current location

```
code/hello.cpp
```

```
../other/file.txt
```

Path shortcuts

```
~ = /home/ujjwal
```

```
. = current folder
```

```
.. = parent folder
```

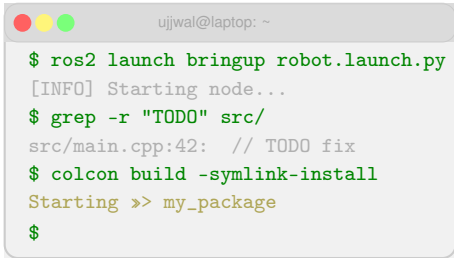
Opening the Terminal

Ctrl + Alt + T

- Or search *Terminal* in the app menu
- Runs a **shell** that interprets commands
- Default shell on Ubuntu: **bash**
- Most tasks are **faster** from terminal than GUI

Why learn the terminal?

SSH into a robot or server = no GUI. The terminal is your only interface.



```
ujjwal@laptop: ~  
$ ros2 launch bringup robot.launch.py  
[INFO] Starting node...  
$ grep -r "TODO" src/  
src/main.cpp:42: // TODO fix  
$ colcon build -symlink-install  
Starting >> my_package  
$
```

Anatomy of the Prompt

```
ujjwal@laptop:~/ros2_ws$
```

ujjwal

Username

laptop

Hostname

~/ros2_ws

Current folder

\$

Prompt

\$ – regular user

Normal everyday work.

– root (superuser)

Never run as root for everyday tasks!

Command Structure

command [-options] [parameters]

command

The program to run. If it is in \$PATH, just type its name.

[-options]

Flags that change behaviour.

Short: -l

Long: --long

Combined: -la

[parameters]

What to act on: files, folders, text strings ...

Command	Options	Parameter	What it does
ls	-la	~	list all files in home (long format)
cp	-r	src/ dst/	copy folder recursively
grep	-r	"TODO" src/	search recursively in src/

Getting Help

```
man <command>
```

Full manual page.

- Arrow keys to scroll
- / to search
- q to quit

```
man ls    man grep    man cp
```

```
command -help / -h
```

Short summary of options. Usually fits one screen.

Long options use - (two dashes):

```
ls -help    git -version
```

Short options use - (one dash):

```
ls -h    colcon -h
```

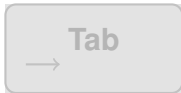
Bonus: tldr (install separately)

```
sudo apt install tldr    then    tldr ls
```

Shows practical examples instead of full documentation. Great for quick lookups.

Tab Completion

The most useful shortcut you will learn today



once: complete

twice: show all matches

Tab completes

- File and folder names
- Command names
- ROS2 topic and node names

Example

Type `cd Dow` then Tab Tab:

`Documents/ Downloads/`

Type `cd Dow` then Tab:

→ completes to `cd Downloads/`

Use Tab constantly

It prevents typos and instantly tells you if a file or command does not exist.

Navigation: pwd and ls

pwd

Print Working Directory.
“Where am I right now?”

```
$ pwd  
/home/ujjwal/ros2_ws
```

ls [options] [path]

List directory contents.

- l long format
- a show hidden files
- h human-readable sizes
- la combine flags

```
$ ls  
src  build  install  log
```

```
$ ls -la ~/ros2_ws  
drwxr-xr-x 4 ujjuwal ujjuwal 80 Sep 10  
    ros2_ws  
drwxr-xr-x 8 ujjuwal ujjuwal 160 Sep 10 src
```

Navigation: cd

Change Directory

Path	Meaning
/	Root of the filesystem
~	Your home folder
.	Current directory
..	Parent directory
-	Previous directory

Tip

`cd -` jumps back to your last directory. Great for switching between two folders quickly.

```
$ cd ~/ros2_ws           # go to ros2_ws in home
$ cd src                 # relative: go into
                        src/
$ cd ..                  # go up one level
$ cd ../..               # go up two levels
$ cd -                   # go back to previous
                        dir
$ cd                      # go home (no argument)
```

Creating Directories: mkdir

```
mkdir [-p] <name>
```

Create a new directory.

-p also creates all missing parent directories. Never fails if the folder already exists.

ROS2 workspace

```
mkdir -p ~/ros2_ws/src
```

Creates both `ros2_ws/` and `src/` in one command.

```
# Create a single folder
```

```
$ mkdir my_package
```

```
# Create nested folders (-p = parents too)
```

```
$ mkdir -p ~/ros2_ws/src/my_package/include
```

```
# Verify
```

```
$ ls ~/ros2_ws/src/
```

```
my_package
```

Copying and Moving

```
cp [-r] <src> <dst>
```

Copy a file or folder.
-r required for directories.

```
mv <src> <dst>
```

Move **or rename** a file.
Works on directories too.
No -r flag needed.

Copy a file

```
$ cp main.cpp backup.cpp
```

Copy an entire folder

```
$ cp -r src/ src_backup/
```

Rename a file

```
$ mv old_name.cpp new_name.cpp
```

Move file to another directory

```
$ mv robot.cpp ~/ros2_ws/src/my_package/
```

Move multiple files at once

```
$ mv *.cpp ~/ros2_ws/src/my_package/
```

Removing Files: rm

```
rm [-r] <name>
```

Remove a file or folder.

-r = recursive (entire folder).

-f = force, no confirmation.

No recycle bin!

Files deleted with `rm` are **gone immediately**. No undo. No trash.

Always `ls` first to see what you are about to delete.

```
# Delete a file
```

```
$ rm old_file.cpp
```

```
# Delete a folder and all its contents
```

```
$ rm -r old_folder/
```

```
# Force delete, no prompts
```

```
$ rm -rf old_folder/
```

```
# Safe workflow: check first!
```

```
$ ls *.o           # see what will be deleted
```

```
$ rm *.o           # then delete
```

Wildcards

Pattern	Matches
*	Any set of chars
?	Exactly one char
[a-f]	Char in range a to f
[^abc]	Char NOT in abc

```
$ ls *.cpp                # all .cpp files
main.cpp  robot.cpp  utils.cpp
```

```
$ ls v0?.pdf              # v01.pdf, v02.pdf, ...
v01.pdf   v02.pdf
```

```
$ ls [uv]01*              # u01.tex, v01.pdf, ...
u01.tex   v01.pdf   v01.tex
```

```
$ rm *.o                  # delete all object
                           files
```

```
$ cp *.h include/         # copy all headers
```

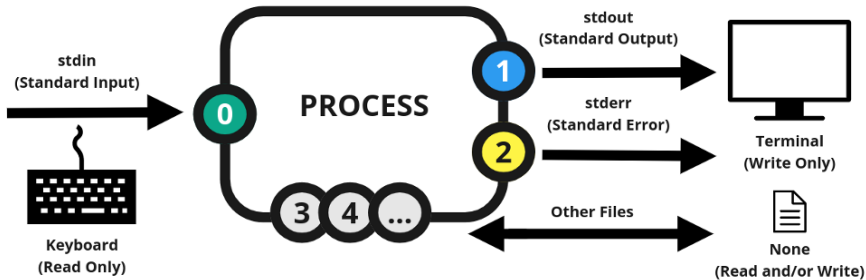
Safe delete workflow

Step 1: `ls *.o`
(check what matches)

Step 2: `rm *.o`
(then delete safely)

Standard I/O Channels

Every program has three default channels



stdin – channel 0

Keyboard by default
`std::cin`

stdout – channel 1

Normal output
`std::cout`

stderr – channel 2

Error messages
`std::cerr`

Redirecting I/O

- `>` redirect, overwrite
- `>>` redirect, append
- `<` feed file as stdin
- `2>&1` merge stderr into stdout
- `/dev/null` – discard all output

```
$ program 1> output.txt      # stdout to file
$ program 2> errors.txt      # stderr to file
$ program 1> out.txt 2> err.txt # separate

$ program 1> all.txt 2>&1 # both to same file
$ program < input.txt      # feed file as stdin

# Append without overwriting
$ echo "hello" >> log.txt

# Discard all output
$ program > /dev/null 2>&1
```

ROS2 log

Save all launch output:

```
ros2 launch bringup robot.launch.py
1>log.txt 2>&1
```



Pipes and Command Chaining

cmd1 | cmd2

stdout of cmd1 becomes stdin of cmd2. Chain as many as needed.

Op	Behaviour
----	-----------

;	run always
&&	run if prev OK
	run if FAILED

Pipe: stdout of left -> stdin of right
\$ ls | grep ".cpp"

Chain multiple programs
\$ cat log.txt | grep ERROR | sort | uniq

; runs next regardless of success
\$ mkdir build ; cd build

&& runs next only if previous succeeded
\$ mkdir build && cd build && cmake ..

|| runs next only if previous FAILED
\$ test -f config.yaml || echo "Missing!"

Viewing and Searching Files

View file contents

- `cat` – print whole file
- `less` – scroll (q to quit)
- `head -n N` – first N lines
- `tail -n N` – last N lines
- `tail -f` – follow live

```
$ cat CMakeLists.txt           # print whole file
$ less build.log               # scroll (q to quit)
$ head -n 20 output.txt        # first 20 lines
$ tail -n 20 output.txt        # last 20 lines
$ tail -f output.txt           # follow live output
```

```
$ grep "TODO" src/main.cpp
$ grep -r "rclcpp::init" src/  # recursive
```

Search

- `grep` – search inside files
- `find` – find files by name

```
$ find ~/ros2_ws -name "*.cpp"
$ find . -name "CMakeLists.txt"
```

Command History

Key / Command	Action
↑ / ↓	scroll history
Ctrl + R	search history
history	show full list
!N	run command N
!!	repeat last
sudo !!	with sudo

```
$ history
421  cd ~/ros2_ws
422  colcon build
423  ros2 run my_pkg my_node
```

```
$ !423          # re-run command 423
$ !!           # re-run last command
$ sudo !!      # re-run last with sudo
```

Ctrl + R search

Press Ctrl+R, start typing. The shell searches backwards through history. Press Enter to run the found command.

Managing Processes

Run in the background

```
$ ros2 run my_pkg my_node &
```

```
[1] 12345
```

Find the process

```
$ ps aux | grep my_node
```

Kill by PID (graceful)

```
$ kill 12345
```

Force kill if it won't stop

```
$ kill -9 12345
```

Kill by name

```
$ killall rviz2
```

Key / Command	Action
Ctrl + C	cancel
Ctrl + Z	suspend
bg	resume in bg
fg	bring to fg
cmd &	start in bg
kill -9	force kill
killall	kill by name
htop	visual viewer

htop

Run htop for a live process manager. Arrow keys to navigate, F9 to kill, q to quit.

Installing Software with APT

APT – Advanced Package Tool

Ubuntu's official package manager.
Downloads from trusted **repositories**.
Always run `apt update` first.

ROS2 packages

```
sudo apt install  
ros-humble-rviz2
```

Libraries: prefix `lib`, suffix `-dev`

```
$ sudo apt update           # refresh  
$ sudo apt upgrade         # upgrade all  
  
$ sudo apt install vim      # install  
$ sudo apt install libopencv-dev  
  
$ apt search opencv        # search  
  
$ sudo apt remove vim      # remove
```

Why Terminal Editors?

When you will need one

- SSH into a robot – no GUI available
- Quick edit of a config or launch file
- GUI broke after a bad update
- Inside Docker or Distrobox
- Editing a file owned by root

Two editors to learn

- **nano** – beginner-friendly. Controls shown on screen. No modes. Works immediately.
- **vim** – fast, powerful. Available on every Linux system. Learning curve, but worth it.

Both already installed

No setup needed on Ubuntu.

nano – Basics

The beginner-friendly terminal editor

```
$ nano ~/.bashrc           # open
$ nano new_script.sh       # new file
$ nano -l config.yaml      # line #s
```

- Just type – no modes to switch
- Arrow keys to move
- [^] means **Ctrl**
- All shortcuts shown **at the bottom**

Beginner workflow

Open → edit → save → exit

```
GNU nano 6.2  ~/.bashrc  Modified

source /opt/ros/humble/setup.bash
source ~/ros2_ws/install/setup.bash
export ROS_DOMAIN_ID=0|

~G Help    ~O Save    ~X Exit    ~W Search
~Z Undo    ~K Cut     ~U Paste   ~/ Goto
```

nano – Key Reference

Shortcut	Keys	Action
<code>^O</code>	Ctrl + O	Save the file (press Enter to confirm)
<code>^X</code>	Ctrl + X	Exit (asks to save if unsaved changes exist)
<code>^W</code>	Ctrl + W	Search – type term, press Enter
<code>^K</code>	Ctrl + K	Cut the current line
<code>^U</code>	Ctrl + U	Paste the cut line
<code>^Z</code>	Ctrl + Z	Undo last change
<code>^/</code>	Ctrl + /	Go to line number
<code>^G</code>	Ctrl + G	Help – full shortcut reference

^ means Ctrl

`^O` = hold Ctrl and press O. You do not type the caret symbol.

vim – Introduction

Vi IMproved – the editor available on every Linux system

- Pre-installed on virtually every Linux system
- Keyboard-only – no mouse needed
- Extremely fast once the muscle memory is built
- **Modal editor** – the same key does different things in different modes

Opening vim

```
vim ~/.bashrc  
vim new_file.py  
vim src/main.cpp
```

Stuck inside vim?

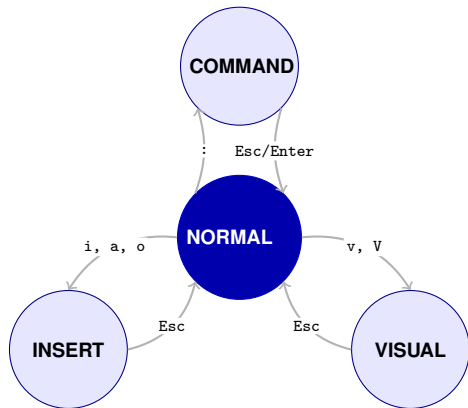
Press Esc, then type :q!, then press Enter.
Exits **without saving**. The most Googled Linux question.

Built-in tutorial

Run vimtutor in the terminal for a 30-minute interactive guide. Strongly recommended.

vim – Modes

Understanding modes is the key to using vim



NORMAL mode

Default. Navigate and run commands. Press **Esc** to always return here.

INSERT mode

Type text. Enter: **i**, **a**, **o**.

VISUAL mode

Select text. Enter: **v** or **V**.

COMMAND mode

Type after **:**. Save, quit, search/replace.

vim – Moving and Editing

All of these work in NORMAL mode

Navigation

- `h j k l` left, down, up, right
- `w` next word `b` back
- `0` start of line `$` end
- `gg` top of file `G` bottom
- `:N` jump to line N

Editing

- `dd` delete (cut) current line
- `yy` copy (yank) current line
- `p` paste below cursor
- `u` undo `Ctrl+R` redo
- `/word` search for word
- `n / N` next / previous match

vim – Saving and Quitting

Press : from NORMAL mode, type the command, press Enter

Command	Entry	Action
:w	Normal → :	Save (write to disk)
:q	Normal → :	Quit (fails if unsaved changes)
:wq	Normal → :	Save and quit
:q!	Normal → :	Quit WITHOUT saving (force)
:set number	Normal → :	Show line numbers
:%s/old/new/g	Normal → :	Replace all occurrences

Insert mode entry

i before a after o new line below

Golden rule

Always press Esc first to return to Normal mode before typing a command.

Why Ubuntu 22.04 May Struggle?

Default kernel 6.8 still struggles with newer Hardware

Common symptoms

- Blank screen after installation
- System freezes when GPU switches
- Wi-Fi not detected at all
- Very poor battery or performance

Quick try: HWE kernel

```
sudo apt install linux-generic-hwe-22.04
```

Upgrades kernel to 6.8. Fixes some hardware – else see the three options below.

Commonly affected hardware

- NVIDIA RTX 40 and 50 series
- AMD Ryzen 7000+ / Ryzen AI 300
- Intel BE200 Wi-Fi adapter
- Mediatek MT7922 Wi-Fi adapter
- Laptops with a MUX switch (hybrid GPU)

Option 1: Ubuntu 24.04 LTS

What changes

- Ships kernel 6.8, **HWE brings it to 7.0**
- Full support for RTX 40/50, Ryzen AI, new Wi-Fi chips
- ROS2 **Jazzy** runs natively

Trade-off

ROS2 **Humble** needs Docker on 24.04.
Course materials target Humble.

Steps

1. Download Ubuntu 24.04 ISO from ubuntu.com
2. Flash to USB (Balena Etcher or Rufus)
3. Install alongside Windows (dual boot)
4. Install ROS2 Humble via Docker

Best for

Post-2022 hardware without a dedicated NVIDIA GPU.

Option 2: Fedora + Distrobox

What is Distrobox?

Runs any Linux distro as a **container** inside your host OS. Shares your home folder, GPU, and hardware. Near-native performance.

- Fedora ships with **kernel 7.1+**
- Distrobox gives full Ubuntu 22.04 inside
- ROS2 Humble runs natively in the container
- Files and hardware work normally

```
# Install on Fedora
```

```
$ sudo dnf install distrobox podman
```

```
# Create Ubuntu 22.04 container
```

```
$ distrobox create -n ros-humble \  
    -i ubuntu:22.04
```

```
# Enter the container
```

```
$ distrobox enter ros-humble
```

```
# Now inside Ubuntu 22.04
```

```
# Install ROS2 Humble normally
```

Best for

All gaming laptops, especially AMD.

Option 3: NVIDIA Laptop – dGPU Only Mode

The problem

Modern NVIDIA laptops have **hybrid graphics**: iGPU for battery life and dGPU for performance. The MUX switch between them causes kernel panics on Ubuntu 22.04.

The fix – in BIOS/UEFI

1. Restart – press Del / F2 / F12
2. Advanced → Display → GPU Mode
3. Set to **dGPU Only** or Discrete GPU
4. Save and exit

Consequences

Higher power use – keep power brick plugged in. Battery life drops noticeably.

Benefit

No driver workarounds needed. Ubuntu 22.04 and ROS2 Humble work cleanly.

Best for

ASUS ROG, MSI, Razer, Lenovo Legion (NVIDIA)

Which Option Should I Choose?

	Ubuntu 22.04	Ubuntu 24.04	Fedora trobox + Dis-	dGPU Only
Kernel	5.15 (HWE: 6.8)	7.0 (HWE)	7.1+	5.15 (HWE: 6.8)
ROS2 Humble	Native	Via Docker	In container	Native
Difficulty	Easy	Easy	Medium	Easy
Overhead	None	None	Minimal	High power
Best for	Works already	Mid hardware	Gaming laptops	NVIDIA laptops

Quick decision guide

Ubuntu 22.04 installs fine → use it.

NVIDIA gaming laptop → try dGPU Only first (easiest).

AMD gaming laptop → Fedora + Distrobox.

Nothing works → Ubuntu 24.04 + Docker.

Thank You

Questions?

Introduction to Linux

Foundation Course – Winter Semester 2026

Ujjwal Patil H-BRS

Further reading and practice

`man bash` `vimtutor` <https://ubuntu.com/tutorials>

